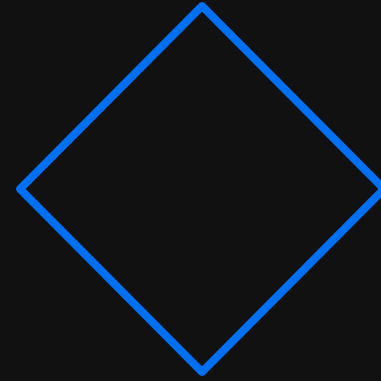


Révision + Next.js



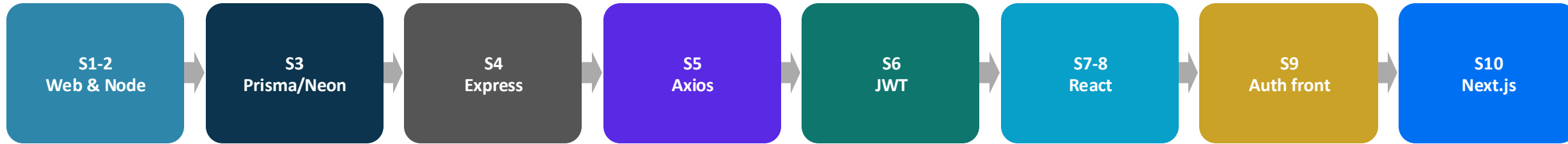
Séance 10 - On révise tout, puis on passe de React + Node à Next.js

Cours 420-941-MA - Collège de Maisonneuve - Groupe 25604

révision générale - full-stack - quiz éclair - transition propre vers Next.js

Bon retour ! La feuille de route

3 semaines de pause : voici le chemin parcouru



Le BACKEND (S1 -> S6)

- Express (API REST) + Prisma/Neon (BD)
- Axios (consommer une API)
- JWT + roles (securite)

Le FRONTEND (S7 -> S9)

- React : composants, JSX, props
- Hooks : useState, useEffect, useContext
- Consommer l'API + auth cote client

Aujourd'hui : on REVISE tout ca ensemble (pour se remettre en selle), PUIS on decouvre Next.js qui va tout reunir.

Objectifs de la séance

Deux temps : réviser, puis démarrer Next.js

1) Révision générale

- La grande image du full-stack
- Backend : Express, Prisma, Axios, JWT
- Frontend : React, hooks, CORS, auth
- Un quiz éclair pour se tester
- -> tout est encore frais et utile

2) Introduction à Next.js

- Pourquoi Next.js (les frictions actuelles)
- React + backend dans UN projet
- Server / Client Components
- Routing par fichiers, Route Handlers
- Créer sa première app Next

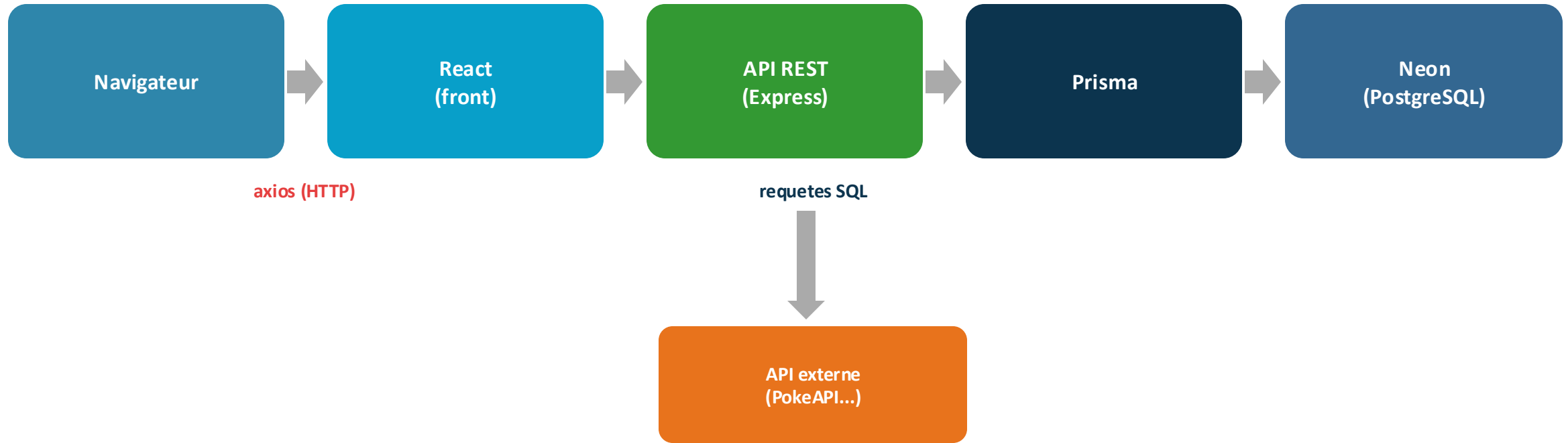
Objectif d'état d'esprit : Next.js n'efface rien de ce qu'on revise aujourd'hui. Il le REORGANISE. Vous connaissez déjà ~80% de ce qu'il faut.

PARTIE 1

Révision générale : la grande image

Comment tout s'assemble

Le voyage d'une requête, de bout en bout



- **Le navigateur affiche React** ; React appelle l'API (axios) pour lire/écrire des données.
- **L'API (Express) applique la logique + la sécurité (JWT)**, puis parle à la BD via Prisma.
- **Le backend peut aussi appeler une API externe** (axios) pour enrichir ses données.

Révision : HTTP + Express

Les routes de l'API

```
app.use(express.json()) // lire le body JSON
```

Express

```
app.get('/livres', (req, res) => res.json([]))
app.get('/livres/:id', (req, res) => {
  const id = Number(req.params.id) // params
})
app.post('/livres', (req, res) => {
  const { titre } = req.body // body
  res.status(201).json({ titre })
})
```

À se rappeler

- Verbes : GET, POST, PUT/PATCH, DELETE
- req.params (chemin), req.query (?..),
- req.body (corps JSON)
- Statuts : 200, 201, 400, 401, 403, 404
- Routers : un fichier par ressource
- Middleware : (req, res, next)

express.json() AVANT les routes, sinon req.body est undefined. Toujours terminer une reponse (res.json / res.status().json / res.end).

Révision : Prisma + Neon

La base de données, sans SQL à la main

```
// schema.prisma : modeles, enums, relations
model Pokemon {
  id          Int    @id @default(autoincrement())
  nom         String
  numeroPokedex Int   @unique
}

// requetes
prisma.pokemon.findMany()
prisma.pokemon.create({ data })
prisma.pokemon.update({ where, data })
```

Prisma

À se rappeler

- `npx prisma migrate dev` (créer/appliquer)
- `npx prisma generate` (le client)
- `npx prisma studio` (voir la BD)
- `findMany` / `findUnique` / `create` /
- `update` / `delete`
- Singleton `utils/prisma.ts`
- Neon = PostgreSQL héberge

Les migrations (`prisma/migrations`) se COMMITTENT sur Git. Le `.env` (secrets) NON.

Révision : Axios

Consommer une API externe

```
// instance + appel
const api = axios.create({ baseURL: '...' })

const { data } = await api.get('/pokemon', {
  params: { limit: 20 },
})
// data est DEJA du JSON (pas de .json())
```

Axios

À se rappeler

- axios.get / post / patch / delete
- reponse.data (deja parse)
- params pour la query string
- leve une erreur sur 4xx/5xx
- -> try/catch + isAxiosError
- instance : baseURL, headers, timeout

Difference avec fetch : axios parse le JSON tout seul (.data) et transforme les statuts d'erreur en exceptions -> plus simple a gerer avec try/catch.

Révision : Authentification (JWT)

Qui es-tu, et qu'as-tu le droit de faire ?

```
// inscription : hacher
const hash = await bcrypt.hash(mdp, 10)

// connexion : comparer + signer
const ok = await bcrypt.compare(mdp, user.password)
const token = jwt.sign({ sub: user.id, role },
    process.env.JWT_SECRET!, { expiresIn: '1h' })

// route protegee : verifier
const payload = jwt.verify(token, process.env.JWT_SECRET!)
```

JWT

À se rappeler

- Mot de passe : bcrypt (jamais en clair)
- Token signe (pas chiffre) : rien de
- sensible dans le payload
- Header : Authorization: Bearer <token>
- authentifier (401) puis exigerRole (403)
- 401 = pas connecte, 403 = pas le droit
- JWT_SECRET dans .env

Révision : React

Composants, JSX, props, listes

```
// un composant = une fonction qui renvoie du JSX
function Carte({ titre }: { titre: string }) {
  return <h3>{titre}</h3>
}

// afficher une liste : .map() + key
<ul>
  {livres.map((l) => <li key={l.id}>{l.titre}</li>)}
</ul>
```

TSX

À se rappeler

- Composant : fonction -> JSX
- props : parametres (parent -> enfant)
- { } : inserer du JS dans le JSX
- .map() + key unique pour les listes
- className (pas class)
- un seul element racine

Révision : les hooks

useState, useEffect, useContext

useState

- `const [v, setV] = useState(x)`
- la MEMOIRE du composant
- `setV(...)` -> re-rendu
- on ne mute jamais v

useEffect

- effet + dependances
- `[]` -> une fois (montage)
- `[x]` -> quand x change
- `return` -> nettoyage
- appels API, minuteurs

useContext

- etat GLOBAL partage
- `createContext` / `Provider`
- `useContext(...)`
- ex : le token (auth)
- evite le prop drilling

```
const [n, setN] = useState(0)
useEffect(() => { chargerDonnees() }, []) // au demarrage
const token = useContext(AuthContext)    // etat global
```

TSX

Révision : front <-> back

Appeler l'API, CORS, token

```
// charger au montage (les 3 etats)
useEffect(() => {
  api.get('/pokedex')
    .then(r => setData(r.data))
    .catch(() => setErreur('...'))
    .finally(() => setChargement(false))
}, [])

// token : dans le header (intercepteur axios)
config.headers.Authorization = `Bearer ${token}`
```

TSX

À se rappeler

- 3 etats : chargement / erreur / succes
- formulaire controle (value + onChange)
- e.preventDefault()
- CORS : protection du navigateur,
- reglee COTE backend (app.use(cors()))
- token : localStorage + header
- AuthContext : etat global connecte

Quiz éclair de révision

Répondez avant de retourner la carte

1. 401 vs 403 ?

-> 401 = pas authentifié ; 403 = authentifié mais pas le droit

2. Que fait [] dans useEffect ?

-> l'effet ne se lance qu'UNE FOIS (au montage)

3. Ou se règle CORS ?

-> côté BACKEND (le serveur autorise le front)

4. Commande qui applique une migration ?

-> npx prisma migrate dev

5. props vs state ?

-> props viennent du parent (lecture) ; state = mémoire locale

6. axios : où sont les données ?

-> dans response.data (déjà en JSON)

PARTIE 2

Et maintenant : Next.js

Notre archi actuelle : ça marche, mais...

Deux projets qui doivent se parler



Les irritants du quotidien :

- **Deux projets a lancer** (deux terminaux, deux npm run dev)
- **Deux deploiements a gerer** (front d'un cote, back de l'autre)
- **CORS a configurer** des que le front appelle le back
- **Les types se dupliquent** (un Pokemon cote back ET cote front)
- **Le 1er affichage est lent** (page vide -> useEffect -> axios -> re-rendu)

Next.js, c'est quoi ?

Le framework React qui inclut le backend

Next.js est un FRAMEWORK bati SUR React. Il garde tout React (composants, JSX, hooks) et ajoute par-dessus : le routing, un backend integre, le rendu cote serveur. Front et back vivent dans UN SEUL projet.

Un seul projet

- Front + back ensemble
- Un seul npm run dev
- Un seul deploiement
- Pas de CORS

Toujours du React

- Composants, JSX
- useState, useEffect...
- props, .map(), key
- Rien a re-apprendre

En plus

- Routing par fichiers
- Rendu cote serveur
- API integree
- Optimise pour la prod

Cree par Vercel, c'est aujourd'hui LE choix par default pour une app React serieuse. Enorme communaute, tres demande sur le marche du travail.

Le concept clé : Server vs Client

Où tourne mon composant ?

Server Component (par défaut)

- Tourne sur le SERVEUR
- Peut lire la BD directement (Prisma !)
- Pas de useState / onClick
- Rapide, bon pour le SEO
- -> pour AFFICHER des données

Client Component ("use client")

- Tourne dans le NAVIGATEUR
- useState, useEffect, onClick
- L'interactivité (boutons, formulaires)
- Ce que vous faisiez déjà en React
- -> pour REAGIR aux actions

```
"use client" // en haut = composant interactif
import { useState } from "react"
export default function Compteur() {
  const [n, setN] = useState(0)
  return <button onClick={() => setN(n + 1)}>{n}</button>
}
```

TSX

Le routing par fichiers

L'arborescence EST le plan du site

```
app/
  layout.tsx      -> gabarit commun
  page.tsx        -> la page /
  lancements/
    page.tsx      -> la page /lancements
    [id]/
      page.tsx    -> la page /lancements/:id
  api/
    pokemons/
      route.ts    -> l'API /api/pokemons
```

dossiers

La regle

- Un dossier = un segment d'URL
-
- page.tsx = une page visible
- route.ts = un point d'API
-
- [id] = parametre dynamique
- (ex. /lancements/42)
-
- Plus besoin de react-router :
- les fichiers font le routing

L'API sans Express : Route Handlers

Vos routes backend, dans le meme projet

Avant - Express :

```
router.get('/', async (req, res) => {  
  const p = await prisma.pokemon  
    .findMany()  
  res.json(p)  
})
```

Express

Après - Next :

```
// app/api/pokemons/route.ts  
export async function GET() {  
  const p = await prisma.pokemon.findMany()  
  return Response.json(p)  
}
```

Next

- **Meme logique, meme Prisma** : seule la 'coquille' change.
- **Le nom de la fonction = le verbe HTTP** (GET, POST, PATCH, DELETE).
- **On renvoie avec** `Response.json(...)` au lieu de `res.json(...)`.
- **L'URL vient du dossier** (`app/api/pokemons` -> `/api/pokemons`).

Charger des données : bien plus simple

Plus besoin de useEffect + axios pour afficher

Avant - React (client) :

```
const [pokemons, setPokemons] =
  useState([])

useEffect(() => {
  axios.get('/pokedex')
    .then(r => setPokemons(r.data))
}, [])
// page vide -> chargement -> re-rendu
```

React

Après - Next (Server Component) :

```
// app/page.tsx (cote serveur)
import prisma from '@lib/prisma'

export default async function Page() {
  const pokemons =
    await prisma.pokemon.findMany()
  return <Liste pokemons={pokemons} />
}
```

Next

un Server Component peut être async et parler à Prisma DIRECTEMENT. Pas de useState, pas de useEffect, pas d'axios, pas de page vide. Les données arrivent déjà prêtes.

React + Express -> Next.js

La table de correspondance

Avant (React + Express)	Avec Next.js
Deux projets (front + back)	UN seul projet
Vite (front) + Express (back)	Next fait les deux
react-router / navigation manuelle	Routing par fichiers (app/)
Routes Express (app.get...)	Route Handlers (app/api/.../route.ts)
useEffect + axios (1er chargement)	fetch cote serveur (Server Component)
CORS a configurer	Meme origine : pas de CORS
Prisma / Neon	Prisma / Neon (IDENTIQUE)
useState, onClick, formulaires	Idem, dans un composant 'use client'

Ce qui reste, ce qui change

Pour dédramatiser

Ce qui NE change PAS

- React (composants, JSX, hooks)
- Prisma + Neon (votre BD)
- La logique métier
- TypeScript
- Les concepts d'auth (JWT) mais Meilleur avec StackAuth + Clerk
- Votre façon de penser en composants

Ce qui CHANGE (à apprendre)

- Routing par fichiers (app/)
- Server vs Client Components
- Route Handlers (au lieu d'Express)
- fetch de données côté serveur
- Quelques conventions Next
- -> ~20% de nouveau, c'est tout

Vous ne repartez PAS de zéro. Vous reorganisez un savoir que vous avez déjà. La courbe est douce.

Créer sa première app Next

Un seul outil, une seule commande

```
npx create-next-app@latest mon-app
# repondre : TypeScript OUI, App Router OUI, Tailwind (au choix)

cd mon-app
npm run dev      # -> http://localhost:3000
```

bash

Structure de depart :

```
mon-app/
  app/
    layout.tsx    <- gabarit commun
    page.tsx      <- page d'accueil /
  public/         <- images, favicon
  package.json
```

tree

Bon a savoir

- Next tourne sur le port 3000
- (comme votre Express avant)
- Rechargement a chaud inclus
- Tout se passe dans app/
- @/ pointe vers la racine (imports)

Une page et une route d'API

Vos deux premiers fichiers Next

Une page (app/page.tsx) :

```
export default function Page() {  
  return (  
    <main>  
      <h1>Bonjour Next.js !</h1>  
    </main>  
  )  
}
```

TSX

Une API (app/api/salut/route.ts) :

```
export async function GET() {  
  return Response.json({  
    message: 'Bonjour depuis l API'  
  })  
}  
// accessible sur /api/salut
```

TSX

- **La page** s'affiche sur `http://localhost:3000/`
- **L'API** répond sur `http://localhost:3000/api/salut`
- **Tout dans le MEME projet** -> la page peut appeler l'API sans CORS
- **On garde Prisma** dans un fichier partage (ex. `lib/prisma.ts`), comme avant

La suite : migrer le Pokédex en Next

Réunir front + back en un seul projet

- 1. Créer l'app Next + brancher Prisma (lib/prisma.ts)
- 2. Route Handlers : /api/pokedex (GET, POST capturer...)
- 3. Une page d'accueil (Server Component) qui affiche les Pokemon
- 4. Un bouton 'Capturer' interactif (Client Component)
- 5. L'authentification, intégrée au projet
- 6. Le déploiement (Vercel) : un seul projet à mettre en ligne

A faire : créer une app Next, afficher une page, écrire une route /api.

Vous connaissez déjà 80%. On assemble le reste ensemble.



Questions ?

Revision faite. Cap sur Next.js !

Vous connaissez déjà 80%. On assemble le reste ensemble.

